GOVERNMENT WELFARE SCHEME AWARENESS GAP MINER

Natural Language Processing and Computational Linguistics (CSI4001)

PROJECT WRITEUP

Fall Semester 2026–2027

Submitted by

YASWANTH CHOWDARY (23MID0051)
RAJENDER KONDRA (23MID0083)
VICKRANT SALUNKE (23MID0084)

VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

School of Computer Science and Engineering, Vellore-632014, Tamil Nadu, India
October, 2026

---

1. Title

Government Welfare Scheme Awareness Gap Miner: identifying awareness-gap signals in public questions and comments about Indian government welfare schemes using text mining and NLP.

2. Motivation

India runs many central welfare schemes, such as PM-KISAN (income support for farmers), PMAY (housing), Ayushman Bharat PM-JAY (health insurance), PMMY/Mudra (micro-credit), MGNREGA (rural employment), PMFBY (crop insurance) and NSAP (pensions). Each has its own eligibility rules, documents and application steps, and eligible citizens often miss benefits because they do not know how to apply, whether they qualify, or why a payment has not arrived.

People who are confused about a scheme leave a written trace: they call helplines, ask questions under scheme-guide articles and post complaints. This text is public and continuous, but it is short, informal, unstructured and often written in Hindi or romanized Hindi (Hinglish), so reading it by hand does not scale. The project turns this text into structured, explainable signals that show *which* scheme people are confused about, *what kind* of information is missing, and *which themes* keep recurring.

3. Existing Gap/Event that Motivated the Project

- **Survey-only evidence.** Field studies across Telangana, Bihar, Maharashtra, Tamil Nadu, Kerala, Meghalaya, Madhya Pradesh, Odisha and Punjab report uneven and often low awareness of welfare schemes, especially in rural areas [1, 2, 3, 4]. These studies use questionnaires and interviews, cover one village or district at a time, and are slow and expensive to repeat.
- **Questions reveal information needs.** Work on farmers' information needs [5] shows that the questions people ask are a usable record of what they do not know, but no tool mined such questions at scale for welfare schemes.
- **Language barrier.** Most text-analysis tools handle English; real public text about schemes is largely Hindi and Hinglish, with spelling variants such as *paisa / paise / paysa*.

- **Question $\neq$ gap.** A plain question ("How do I apply?") is an information intent, not proof that information is missing. Existing approaches do not separate the two, and they force a label even when a model is unsure.

The project addresses these gaps with a multilingual pipeline that labels every record with evidence, separates intent from awareness gap, and marks uncertain cases as *unclear* instead of guessing. Results describe the analysed text only, not awareness in the whole population.

4. Architecture

Figure 1 shows the system. It has four layers:

1. **Inputs:** public web pages (RSS, sitemaps, WordPress comments, FAQs, PDFs), the Kisan Call Centre open-data API, and CSV/JSON dataset uploads through the web interface.
2. **Collection engine (background worker):** runs collection jobs with budgets, heartbeat and resume after a crash. A *SafeFetcher* allows public addresses only (SSRF guard), obeys robots.txt, waits at least two seconds between requests to the same host and caps responses at 5 MB. Extractors pull comments, FAQs, feed items, PDFs and API records with their own publication dates.
3. **Ingest pipeline:** one pipeline for both live and uploaded data: validate and normalise, redact personal identifiers, remove duplicates, embed, run NLP analysis, then store and cluster.
4. **Store and serve:** versioned model artefacts, a SQLite database (PostgreSQL configuration provided), a FastAPI back end with REST and live Server-Sent Events, and a Next.js dashboard.

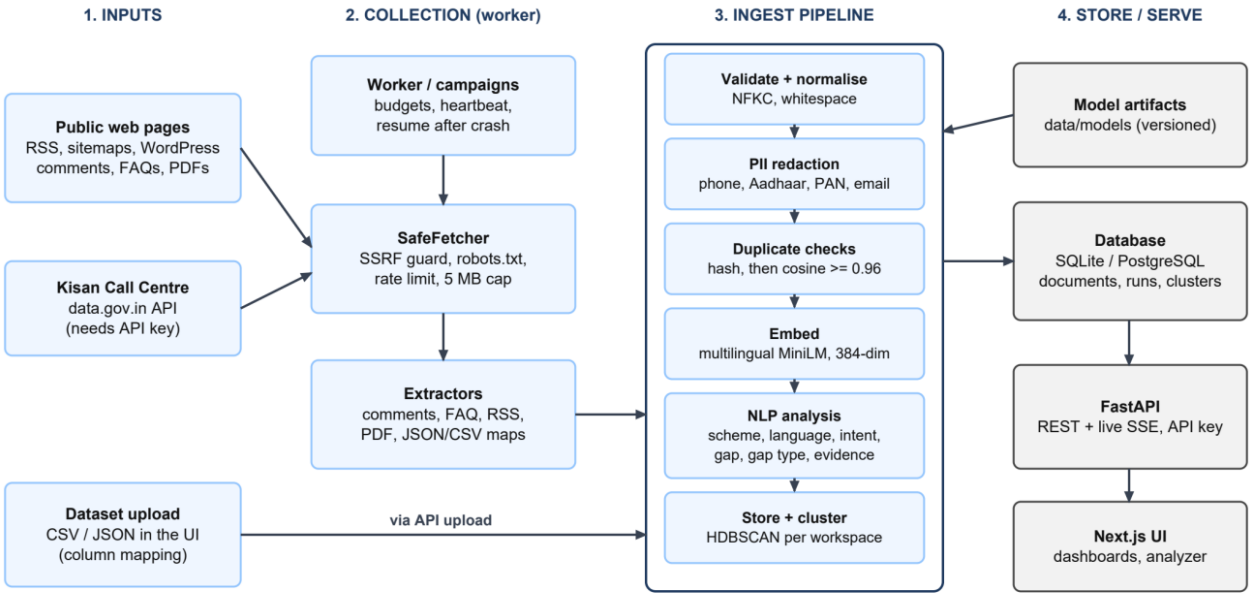


Figure 1: End-to-end architecture. Dataset uploads skip the collection stage; both paths share the same ingest pipeline and database.

Data model. The central `documents` table holds the cleaned text, source, timestamps, scheme, language, labels and evidence. Collection jobs, crawled pages, events, embeddings, clusters and model versions link to it. All timestamps are stored in UTC.

5. Complete Techstack Info

Layer	Technology and role
Frontend	Next.js 16, React 19, Tailwind CSS 4: dashboards, record explorer, clusters, geography, live collection monitor, text analyzer, semantic search, models page. Filters are stored in the URL.
Back end / API	Python, FastAPI (REST + Server-Sent Events), SQLAlchemy 2 (ORM), Alembic (migrations). API-key authentication for all write operations, rate limiting, upload limits and an audit trail.
Worker	Python background worker (`backend.worker`): runs collection jobs, picks up pending NLP jobs, sends heartbeats and resumes stalled jobs.
Database	SQLite (used for all results); PostgreSQL 16 configuration via Docker Compose.
Data collection	httpx (HTTP client), BeautifulSoup and lxml (HTML parsing), feedparser (RSS/Atom), pypdf (PDF text).
NLP and ML	sentence-transformers (`paraphrase-multilingual-MiniLM-L12-v2`, 384-d), scikit-learn (TF-IDF, logistic regression, PCA, metrics), hdbscan (clustering), langdetect (language detection), Python `re` (scheme rules and PII redaction), `hashlib` SHA-256 (exact duplicates).
Testing and tooling	pytest (133 back-end tests, mock HTTP only), frontend lint, type-check and build; npm scripts for migrate, seed, train, reprocess and backup.
Deployment	Local development machine (npm scripts start the UI, API and worker together); Docker Compose file for PostgreSQL, migrate, api, worker and web services.

6. Datasets Used

Two workspaces are kept separate so that analytics never mix them.

(a) Labelled training dataset. 127 manually labelled English records (29 of them synthetic augmentations), extended with 126 Hindi and 127 romanized-Hindi translations and 252 labelled live comments, giving 632 labelled records. A record and its translations always stay in the same split, so the test set (101 records; 51 for gap type) contains no translated copy of a training record.

(b) Live collected data. 3,329 unique records (620 duplicates excluded) from permitted public sources:

Source	Type	Records
SarkariYojana – latest posts (RSS)	RSS feed and reader comments	2,453
Kisan Call Centre (data.gov.in API) [6]	Official API, government-scheme queries	199
SarkariYojana – scheme guides	Reader comments on guide pages	25
PM Vishwakarma portal and others	Official FAQ pages and HTML	26

Kisan Call Centre records include state, district and call time; web comments carry their own publication time but no location. Portals that block automated requests, have broken TLS certificates or load content only through JavaScript are recorded as failures, never bypassed.

7. List of Prompts Used for the Construction of the Website

Not applicable.

8. NLP Pipeline

The same text preparation function is used at training and prediction time, so the model never sees differently prepared input. Figure 2 follows one romanized-Hindi comment through the pipeline.

1. **Normalisation:** Unicode NFKC, removal of invisible characters and extra whitespace; model input is lower-cased with URLs removed.
2. **PII redaction:** regular expressions replace phone, Aadhaar, PAN, e-mail, PIN-code and long ID numbers with tags before storage.
3. **De-duplication:** SHA-256 hash for exact copies; cosine similarity ≥ 0.96 between embeddings for near-copies. Call-centre records are de-duplicated by call ID instead.
4. **Language detection:** share of Devanagari characters, then a romanized-Hindi word list, then langdetect, whose answer is accepted only if plausible for Indian sources.
5. **Scheme detection:** rule-based; each of the 15 schemes has strong and weak aliases matched as word-bounded regular expressions. Unmatched records are *Other / Unknown*.
6. **Sentence embeddings:** multilingual MiniLM [7, 8] maps each record to a 384-d unit vector, so English, Hindi and Hinglish sentences with the same meaning lie close together. Similarity is $\text{sim}(a,b) = a \cdot b$.
7. **Classification (Fig. 3):** three tasks: awareness gap (yes/no), intent (8 classes) and gap type (7 classes). Candidates per task: majority baseline, TF-IDF [9] word 1–2 grams + character 2–5 grams with logistic regression, logistic regression on embeddings, and an ensemble that averages the two: $P_{ens}(c|x) = \frac{1}{2}\big(P_{tfidf}(c|x) + P_{emb}(c|x)\big)$. Classes are weighted; the best on validation macro-F1 is retrained and tested once.
8. **Uncertainty handling (Fig. 4):** a gap label is given only to public-voice text, when the nearest labelled example has similarity $s_{max} \geq 0.423$ and confidence $\max(p, 1-p) \geq 0.55$; otherwise the record is *unclear*. News, FAQ and article text is *not applicable*.
9. **Evidence:** each decision stores the model probability, a reason and the three most similar labelled examples, so a person can check every label. Operator corrections always override the model.
10. **Clustering and analytics:** PCA to 20 dimensions, HDBSCAN [10] clustering (scikit-learn [11] for PCA and the classifiers), class-based TF-IDF keywords per cluster, and a gap rate $= \text{yes}/(\text{yes}+\text{no})$ shown with its denominator.

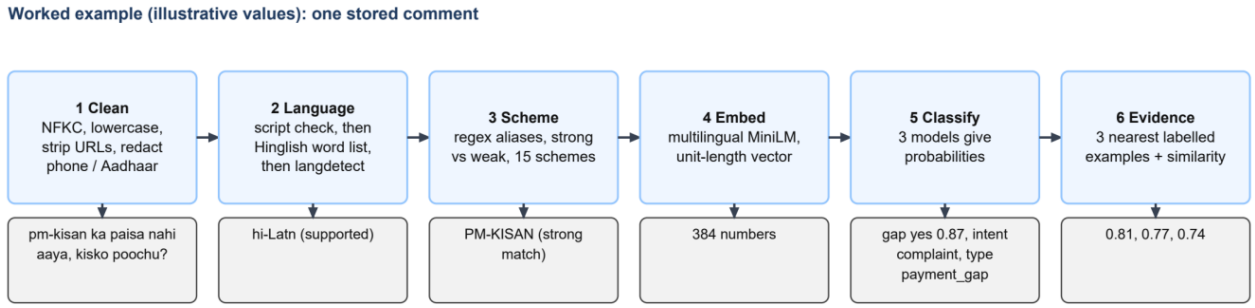


Figure 2: Processing of one record (illustrative values).

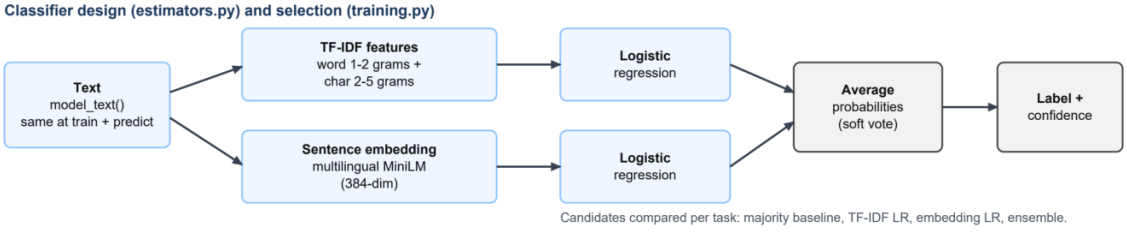


Figure 3: Classifier design: TF-IDF and embedding models combined by soft voting.

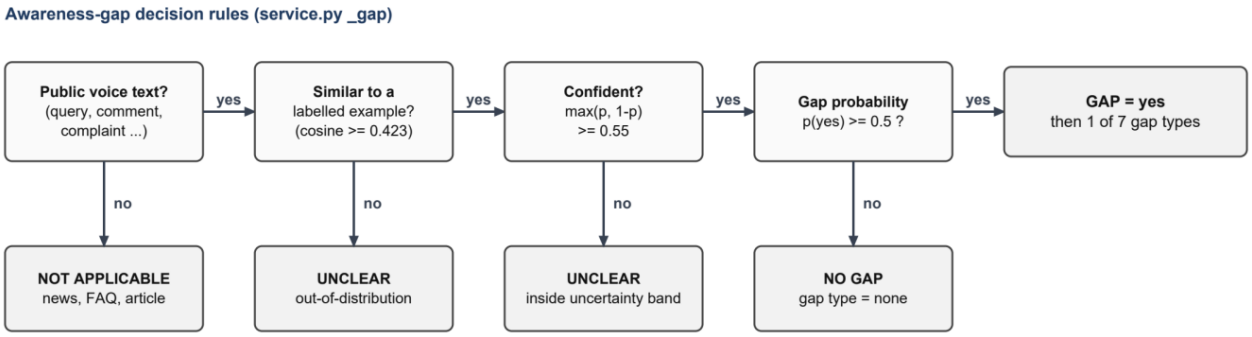


Figure 4: Awareness-gap decision rules.

Results. On 101 held-out test records the awareness-gap model (embedding + logistic regression) reached a macro-F1 of 0.65 (accuracy 0.653) against 0.34 for the majority baseline; intent reached 0.43 and gap type 0.35 (ensembles). Of 3,329 live records, 385 were gap signals, 2,170 no gap, 482 unclear and 292 not applicable, a gap rate of 15% over 2,555 decided records. HDBSCAN found 10 live clusters (silhouette 0.43), with themes such as PMAY list status, Ayushman card eligibility and PM-KISAN instalment status.

9. Users, Inputs (Types, Formats) and Output

Users

- **Viewers / analysts** (policy researchers, NGO staff, scheme administrators): read dashboards, explore records, clusters and geography, and run the text analyzer and semantic search.
- **Operators:** hold the operator API key; start collection jobs, register sources, upload datasets, correct labels and retrain models.

Inputs

Input	Format	Details
Live web sources	HTML, RSS/Atom, sitemaps, PDF	Reader comments, FAQs, feed items; only public pages allowed by robots.txt; a record is kept only if it names a scheme.
Kisan Call Centre	JSON via data.gov.in REST API	Filtered to government-scheme queries; includes state, district and call time; needs an API key.
Dataset upload	CSV or JSON file	Uploaded in the UI with preview and column mapping (text, labels, optional scheme, location, date).
Ad-hoc text	Plain text (English, Hindi, Hinglish)	Typed into the text analyzer for an instant prediction.
Search query	Plain text	Matched by meaning (cosine ≥ 0.35), not exact words.
Label corrections	Form input in record view	Operator fixes override the model and survive reprocessing.

Outputs

- **Per-record labels:** scheme, language (en / hi / hi-Latn), intent (8 classes), awareness gap (yes / no / unclear / not applicable), gap type (7 classes or none), probabilities, confidence flag, reason and three nearest labelled examples.

- **Analytics:** outcome counts, gap rate with its denominator, per-scheme tables, timelines and a state-level view built only from source-stated locations.
- **Clusters:** recurring themes with keywords and example records.
- **Semantic search results** and **text-analyzer predictions**.
- **Model evaluation page:** selected models, test metrics and per-slice results.
- **Collection monitoring:** live job progress streamed over SSE, job history, per-source results and event logs.

10. References

[1] R. R. Bogam, V. M. Chavan and C. U. Rani, "Awareness and utilization of social welfare schemes among elderly villagers of Mahabubnagar rural region in Telangana State of India – An interventional study approach," *J. Family Med. Prim. Care*, vol. 12, no. 9, pp. 2058–2063, 2023, doi:10.4103/jfmpc.jfmpc_753_23.

[2] H. S. Shyam, S. Mishra, D. Dutt, S. Mehra and A. R. Dubey, "Measuring rural people awareness and access to various government schemes – A baseline study in six villages of Buxar District of Bihar," *Int. J. Manag. Issues Res.*, vol. 2, no. 2, pp. 81–86, 2013.

[3] P. Kaviya, N. Manivannan and M. Natarajan, "Knowledge level of tribal beneficiaries towards Pradhan Mantri Kisan Samman Nidhi (PM-KISAN) scheme," *Int. J. Social Sciences*, vol. 13, no. 3, pp. 135–137, 2024, doi:10.46852/2249-6637.03.2024.5.

[4] E. Joseph and N. Vikraman, "Agri-entrepreneur's awareness on government schemes," *J. Krishi Vigyan*, vol. 12, no. 4, pp. 737–742, 2024, doi:10.5958/2349-4433.2024.00128.X.

[5] V. Nikam, A. Ashok and S. Pal, "Farmers' information needs, access and its impact: Evidence from different cotton producing regions in the Maharashtra state of India," *Agricultural Systems*, vol. 196, Art. 103317, 2022, doi:10.1016/j.agsy.2021.103317.

[6] Government of India, "Kisan Call Centre (KCC) query data," Open Government Data Platform India, data.gov.in.

[7] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. EMNLP-IJCNLP*, 2019, pp. 3982–3992.

[8] N. Reimers and I. Gurevych, "Making monolingual sentence embeddings multilingual using knowledge distillation," in *Proc. EMNLP*, 2020, pp. 4512–4525.

[9] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Inf. Process. Manag.*, vol. 24, no. 5, pp. 513–523, 1988.

[10] R. J. G. B. Campello, D. Moulavi and J. Sander, "Density-based clustering based on hierarchical density estimates," in *Advances in Knowledge Discovery and Data Mining (PAKDD)*, Springer, 2013, pp. 160–172.

[11] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.